

LAB PROGRAMS

CSA0623 DAA

Name: V.KAVIYA

Reg No:192421022

Experiment 26: Number of Ways to Move Ball Out of Grid (DP)

main.py	Output
<pre>1 def findPaths(m, n, N, i, j): 2 dp = [[0] * n for _ in range(m)] 3 dp[i][j] = 1 4 result = 0 5 6 for step in range(N): 7 temp = [[0] * n for _ in range(m)] 8 9 for r in range(m): 10 for c in range(n): 11 if dp[r][c] > 0: 12 for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]: 13 nr = r + dr 14 nc = c + dc 15 16 if nr < 0 or nr >= m or nc < 0 or nc 17 >= n: 18 result += dp[r][c] 19 else: 20 temp[nr][nc] += dp[r][c] 21 22 dp = temp 23 return result 24 print(findPaths(2, 2, 2, 0, 0))</pre>	<pre>6 === Code Execution Successful ===</pre>

Experiment 27: House Robber II (Circular Houses)

main.py	Output
<pre>1 def rob_linear(nums): 2 prev = curr = 0 3 4 for money in nums: 5 prev, curr = curr, max(curr, prev + money) 6 return curr 7 8 def rob(nums): 9 if len(nums) == 1: 10 return nums[0] 11 return max(rob_linear(nums[:-1]), rob_linear(nums[1:])) 12 13 nums = [2, 3, 2] 14 print("Maximum Money:", rob(nums))</pre>	<pre>Maximum Money: 3 === Code Execution Successful ===</pre>

Experiment 28: Climbing Stairs

main.py	Output
<pre>1 def climbStairs(n): 2 if n <= 2: 3 return n 4 5 a = 1 6 b = 2 7 for i in range(3, n + 1): 8 a, b = b, a + b 9 10 return b 11 print(climbStairs(4))</pre>	5 === Code Execution Successful ===

Experiment 29: Unique Paths in Grid

main.py	Output
<pre>1 def uniquePaths(m, n): 2 dp = [[1] * n for _ in range(m)] 3 4 for i in range(1, m): 5 for j in range(1, n): 6 dp[i][j] = dp[i-1][j] + dp[i][j-1] 7 8 return dp[m-1][n-1] 9 10 print(uniquePaths(7, 3))</pre>	28 === Code Execution Successful ===

Experiment 30: Large Group Positions

main.py	Output
<pre>1 def largeGroupPositions(s): 2 result = [] 3 start = 0 4 5 for i in range(len(s)): 6 if i == len(s)-1 or s[i] != s[i+1]: 7 if i - start + 1 >= 3: 8 result.append([start, i]) 9 start = i + 1 10 return result 11 12 s = "abbxxxxzzy" 13 print(largeGroupPositions(s))</pre>	[[3, 6]] === Code Execution Successful ===

Experiment 31: Game of Life.

main.py		Output
<pre> 1 board = [2 [0,1,0], 3 [0,0,1], 4 [1,1,1], 5 [0,0,0]] 6 7 rows = len(board) 8 cols = len(board[0]) 9 new = [[0]*cols for _ in range(rows)] 10 11 for i in range(rows): 12 for j in range(cols): 13 count = 0 14 15 for x in range(max(0, i-1), min(rows, i+2)): 16 for y in range(max(0, j-1), min(cols, j+2)): 17 if (x != i or y != j) and board[x][y] == 1: 18 count += 1 19 20 if board[i][j] == 1: 21 if count == 2 or count == 3: 22 new[i][j] = 1 23 else: 24 if count == 3: 25 new[i][j] = 1 26 27 print("Next State:") 28 for row in new: 29 print(row) </pre>		Next State: <pre> [0, 0, 0] [1, 0, 1] [0, 1, 1] [0, 1, 0] </pre> === Code Execution Successful ===

Experiment 32: Champagne Tower

main.py		Output
<pre> 1 poured = 2 2 row = 1 3 glass = 1 4 5 dp = [[0] * 101 for _ in range(101)] 6 dp[0][0] = poured 7 8 for i in range(row + 1): 9 for j in range(i + 1): 10 extra = max(0, (dp[i][j] - 1) / 2) 11 dp[i + 1][j] += extra 12 dp[i + 1][j + 1] += extra 13 14 print(min(1, dp[row][glass])) </pre>		0.5 === Code Execution Successful ===